

PostgreSQL: Advanced Internals and Programmability

Advanced Databases

M.Sc. Tran Quang Toan

Faculty of Data Science and Artificial Intelligence

Bases and coordinates

1. The Shift: From MySQL to PostgreSQL
2. Setup & Environment Preparation
3. Advanced Data Types & Syntax
4. Programmability: Views, Functions & Triggers
5. Final Review Quiz

Learning Objectives

After this lecture, students will be able to:

- Differentiate between **PostgreSQL** and **MySQL** in terms of data integrity, SQL standards compliance, and extensibility.
- Install and configure a PostgreSQL environment using **pgAdmin 4** and restore professional sample databases (DVD Rental).
- Implement advanced data types such as ARRAY, UUID, and JSONB to solve modern data modeling challenges.
- Build reusable database logic using Materialized Views, PL/pgSQL Functions, and Triggers.
- Contrast the **reusability** of stored objects between PostgreSQL and MySQL to improve database design patterns.

The Shift: From MySQL to PostgreSQL

Why PostgreSQL?

The Big Question

You have mastered the basics with **MySQL** in your first year. Why do we need to learn **PostgreSQL** now?

Key reasons for the shift:

- *Industry Demand*: Powering critical systems at Apple, Instagram, and Spotify.
- *Academic Rigor*: Closer to relational theory and official SQL standards.
- *Advanced Features*: Handling complex data types (JSONB, Arrays, GIS) that MySQL struggles with.
- *Architectural Depth*: Understanding how a high-performance engine truly works.

PostgreSQL vs. MySQL: Data Integrity

Data Integrity is the most significant philosophical difference.

- **MySQL (The "Flexible" approach):**
  - Historically more "forgiving."
  - Might perform *implicit type casting* or *silent truncation* (e.g., inserting a long string into a VARCHAR(10) in some modes).
- **PostgreSQL (The "Strict" approach):**
  - Strict type checking. If data doesn't match the schema → **ERROR**.
  - *"Fail fast, fail early"* – Ensures that invalid data never touches the disk.

Pro-Tip

Postgres treats your data like a **Banker** (safety first), while MySQL often treats it like a **Web Developer** (speed/flexibility first).

Compliance and Extensibility

1. **SQL Standards**
   - **SQL:2023 Compliance:** Postgres supports **160 out of 179** core mandatory features.
   - MySQL follows its own dialect, making migrations harder.

2. **Object-Relational**
   - Postgres is not just a Relational DB.
   - It is **Object-Relational**, meaning you can define your own:
     - Data Types
     - Functions
     - Index Types

In Postgres, the database is an extensible platform, not just a storage box.

MySQL vs. PostgreSQL: A Direct Comparison

Feature	MySQL	PostgreSQL
Philosophy	Focus on read speed, simplicity, and Web popularity.	Focus on data integrity and complex feature sets.
SQL Standard	Partially compliant (high flexibility).	Highly compliant (very strict).
Data Types	Basic (Int, Varchar, Date, etc.).	Rich (JSONB, Array, Range, Geometric).
Concurrency	Row-level locking (InnoDB).	**MVCC** (Optimized for concurrent Read/Write).

Setup & Environment Preparation

Installation Guide

- **PostgreSQL Engine (v16):**
  - Official binaries: *enterprisedb.com/downloads*
  - Industry-standard relational database server.
- **pgAdmin 4 (GUI):**
  - The most popular Open Source administration and management tool for PostgreSQL.
  - Included in the standard installer for Windows/macOS.

Video Tutorial

Scan or click here for step-by-step installation:

https://www.youtube.com/watch?v=GpqJzWCcQXY

*Note: Remember your **superuser password** (postgres) and the **port** (default is 5432).*

The Sample Database: DVD Rental

We will use the **dvdrental** database, which represents the business processes of a DVD rental store (similar to the classic "Sakila" in MySQL).

- **Download Link:** *neon.com/postgresql-sample-database*
- **Format:** Download the .zip file and extract it to get the .tar file.

Why this database?

- Contains 15 tables (Films, Actors, Customers, etc.).
- Perfect for practicing complex Joins, Views, and Triggers.
- Rich data history for performance testing.

How to Import Data (pgAdmin 4)

To restore the dvdrental.tar file, follow these 4 steps:

1. **Create Database:** Right-click on *Databases* → *Create* → *Database...* (Name it: dvdrental).
2. **Open Restore Dialog:** Right-click on the newly created dvdrental database → **Restore**.
3. **Select File:** In the "Filename" field, select your dvdrental.tar file (Note: Change file filter to "All Files" to see it).
4. **Execute:** Click the **Restore** button and wait for the "Process Completed" notification.

Troubleshooting

If the "Restore" button is disabled, ensure you have set the *Binary Path* in pgAdmin 4 Settings to your PostgreSQL bin folder.

Advanced Data Types & Syntax

The ARRAY Data Type

In MySQL, you usually need a separate table for 1-N relationships. In Postgres, you can use **ARRAY** for simple collections.
When to use? Small, fixed-size lists (e.g., tags, phone numbers, special attributes).

Hands-on: Adding Tags to Films

```sql
-- 1. Add a tags column to the film table
ALTER TABLE film ADD COLUMN tags TEXT[];

-- 2. Update a film with an array of tags
UPDATE film
SET tags = ARRAY['Action', 'Classic', 'New']
WHERE film_id = 1;

-- 3. Query films that have 'Action' in their tags
SELECT title, tags FROM film
WHERE 'Action' = ANY(tags);
```

Advantage: Simpler queries without extra JOINs. Disadvantage: Harder to enforce referential integrity on elements.

UUID (Universally Unique Identifier)

Why move away from Integer IDs?

- **Security:** Integer IDs (1, 2, 3...) allow "ID Enumeration" (e.g., a competitor can guess your total orders by looking at the URL).
- **Distributed Systems:** You can generate a UUID on a mobile app without asking the server for the next ID.
- **Merging Data:** No ID conflicts when merging two databases.

UUID: Secure Distributed Identifiers

Goal: Replace predictable Integer IDs with globally unique 128-bit identifiers.

```sql
-- 1. Enable extension for UUID functions
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- 2. Create table using UUID as Primary Key
CREATE TABLE secure_orders (
    order_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    customer_id INT REFERENCES customer(customer_id),
    order_date TIMESTAMP DEFAULT NOW()
);

-- 3. Insert and get the generated ID back
INSERT INTO secure_orders (customer_id)
VALUES (1)
RETURNING order_id;

-- Example Result: 'a0eebc99-9c0b-4ef8-bb6d-6bb9bd380a11'
```

Implementing UUID in Postgres

- **Benefit:** Prevents "ID Enumeration" attacks.
- **Compatibility:** Fully supported by modern ORMs.

JSONB (Binary JSON)

PostgreSQL is often called the **"Best NoSQL database"** because of JSONB.

- **Postgres (JSONB) vs. MySQL (JSON):** Postgres stores JSON in a decomposed binary format. It's slightly slower to write but **much faster** to process and supports **GIN Indexing**.

Hands-on: Storing Extra Metadata

```
-- Add a metadata column for technical specs
ALTER TABLE film ADD COLUMN stats JSONB;

UPDATE film SET stats = '{"resolution": "4K", "fps": 60, "hdr": true}'
WHERE film_id = 1;

-- Query: Find films where resolution is 4K
SELECT title FROM film
WHERE stats->>'resolution' = '4K';
```

Programmability: Views, Functions & Triggers

Views vs. Materialized Views

MySQL only has standard Views. Postgres offers **Materialized Views** which store data physically for extreme performance.

```sql
-- Regular View (Computes on every query)
CREATE VIEW film_stats AS
SELECT f.title, COUNT(r.rental_id) as total_rentals
FROM film f JOIN inventory i ON f.film_id = i.film_id
JOIN rental r ON i.inventory_id = r.inventory_id
GROUP BY f.title;

-- Materialized View (Caches results to disk)
CREATE MATERIALIZED VIEW film_stats_cached AS
SELECT * FROM film_stats;

-- To update the data (Manual refresh required)
REFRESH MATERIALIZED VIEW film_stats_cached;
```

Creating a Materialized View for Rental Stats

Benefit: Ideal for complex reports in ‘dvdrental‘ that don’t need real-time data but need to be **lightning fast**.

Deep Dive: View vs. Materialized View

Regular View (Virtual)

- **Storage:** Stores only the *query definition*.
- **Execution:** Every time you query the View, Postgres runs the JOINs and GROUP BY from scratch.
- **Data:** Always 100% up-to-date.
- **Cost:** High CPU/Memory usage for large tables.

Materialized View (Physical)

- **Storage:** Stores the **actual result set** on the disk (like a table).
- **Execution:** Queries are near-instant because it reads pre-computed data.
- **Data:** Can become "stale" (outdated) until REFRESH is called.
- **Cost:** Uses disk space but saves CPU/Time.

The Trade-off: Performance vs. Data Freshness.

When to Refresh Materialized Views?

Unlike regular Views, Materialized Views do not update when the base tables change. You must decide **when** to refresh:

1. **On-Demand (Manual):**
   - When an admin or manager needs the latest report.
   - *Use case:* Monthly financial summaries.
2. **Scheduled (Cron Jobs):**
   - Refresh every hour, every night, or every 15 minutes.
   - *Use case:* A dashboard showing "Top Renting Customers" updated once an hour.
3. **Trigger-Based (Advanced):**
   - Call `REFRESH` inside a Trigger whenever the base table is modified.
   - *Caution:* This can slow down every INSERT/UPDATE (defeats the performance purpose).

Functions: MySQL vs. PostgreSQL

MySQL Approach

```mysql
DELIMITER //
CREATE FUNCTION get_fee(id INT)
RETURNS DECIMAL(5,2)
DETERMINISTIC
BEGIN
  DECLARE fee DECIMAL(5,2);
  -- Logic here...
  RETURN fee;
END //
DELIMITER ;

```

Cons: Must change delimiters; limited return types.

PostgreSQL Approach

```postgresql
CREATE FUNCTION get_fee(id INT)
RETURNS NUMERIC AS $$
DECLARE
  fee NUMERIC;
BEGIN
  -- Logic here...
  RETURN fee;
END;
$$ LANGUAGE plpgsql;

```

*Pros: **Dollar Quoting** ($$) is cleaner, supports returning **TABLE** or **SETOF**.*

Why is Postgres ”More Powerful” for Developers?

- **The Syntax Advantage ($ $):**
  - In MySQL, delimiters are a client-side hack.
  - In Postgres, **Dollar Quoting** makes the code part of the SQL standard string literal. It’s cleaner, safer, and allows nesting functions inside functions easily.

- **The Power Advantage (Complex Types):**
  - **MySQL:** Returns a value (The ”What”).
  - **Postgres:** Returns a structure (The ”Everything”).
  - You can use a Postgres function in the FROM clause of another query. This is called a **Table-Valued Function**, making your SQL incredibly modular.

Procedures: Transaction Control

The Main Difference: Can we control transactions inside the code?

Feature	MySQL Procedure	PostgreSQL Procedure (v11+)
Call Method	`CALL proc_name();`	`CALL proc_name();`
Transactions	Usually depends on the caller's session.	**Internal COMMIT/ROLLBACK** supported.
Usage	General logic grouping.	High-integrity batch processing (e.g., Bank transfers).

```
1  CREATE PROCEDURE transfer_funds(acc_from INT, acc_to INT, amount NUMERIC)
2  AS $$
3  BEGIN
4      UPDATE accounts SET balance = balance - amount WHERE id = acc_from;
5      UPDATE accounts SET balance = balance + amount WHERE id = acc_to;
6
7      -- This is NOT possible in MySQL Functions or Postgres Functions:
8      IF amount > 10000 THEN
9          COMMIT; -- Save point
10     END IF;
11 END;
12 $$ LANGUAGE plpgsql;
```

Postgres-only Feature: Internal Transaction

Triggers: Postgres vs. MySQL

In MySQL, logic is inside the Trigger. In Postgres, you create a **Trigger Function** first, then the **Trigger**.

```
-- 1. Define the Function
CREATE OR REPLACE FUNCTION update_timestamp()
RETURNS TRIGGER AS $$
BEGIN
    NEW.last_update = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- 2. Define the Trigger (Reusable on any table!)
CREATE TRIGGER trg_update_customer_time
BEFORE UPDATE ON customer
FOR EACH ROW EXECUTE FUNCTION update_timestamp();
```

Automatic Last Update Timestamp

You can attach the same $\mathrm{update_timestamp()}$ function to 100 different tables. In MySQL, you must rewrite (copy-paste) the logic for every single trigger.

Trigger Architecture: MySQL vs. PostgreSQL

MySQL: ”Tightly Coupled”

- Logic is **embedded** inside the trigger.
- One trigger = One logic block.
- Hard to maintain if 50 tables need the same audit logic.

Copy-Paste approach

Postgres: ”Decoupled/Modular”

- **Step 1:** Define a standalone **Trigger Function**.
- **Step 2:** Bind that function to any table via a **Trigger**.
- Huge advantage for DRY (Don’t Repeat Yourself) principle.

Plug-and-Play approach

Understanding Special Trigger Variables

Since a Trigger Function is standalone, Postgres provides **automatic variables** to let the function know its context:

- NEW: A record variable holding the **new data row** (for INSERT/UPDATE operations).
- OLD: A record variable holding the **old data row** (for UPDATE/DELETE operations).
- TG_OP: A string indicating the operation: 'INSERT', 'UPDATE', or 'DELETE'.
- TG_TABLE_NAME: A string containing the name of the table that fired the trigger.

The "Return" Mystery

- **RETURN NEW;** → Proceed with the operation using the modified row.
- **RETURN NULL;** → Cancel the operation for the current row (Skip the Insert/Update).

Handling ”Bad” Data: MySQL vs. PostgreSQL

MySQL (Query Level)

- Use `INSERT IGNORE`.
- Hardcoded behavior: ignores *all* basic errors.
- Logic is in the application’s SQL string.

Postgres (Database Level)

- Use `RETURN NULL` in Trigger.
- **Smart Filtering**: You define exactly which rows to skip and which to fail.
- Logic is **centralized** in the DB, safe for all applications.

The Philosophy

MySQL focuses on **flexibility** (Just get the data in).
PostgreSQL focuses on **programmable integrity** (Only get the *right* data in).

Comparison: Postgres vs. MySQL Programmability

Feature	MySQL	PostgreSQL
Language	Limited SQL-based	**PL/pgSQL**, Python, Perl, C
Transactions	No COMMIT in Functions	**COMMIT/ROLLBACK** in Procedures
Trigger	Logic bound to 1 table	Logic in **Reusable Functions**
Views	Dynamic only	**Materialized Views** (Indexed)

Final Review Quiz

1. **Advanced Types:** In the $\mathrm{dvdrental}$ database, if we want to store multiple phone numbers for a staff member without creating a new table, which data type should we use?
2. **Performance:** You have a complex query for a monthly report that takes 30 seconds to run. You don't need real-time data. Which PostgreSQL feature would you use to make it run in milliseconds?
3. **Programmability:** What is the main structural difference between creating a Trigger in MySQL versus PostgreSQL?